```
#           to standard output
```

Labeled lists may also be produced by putting a double colon after the label. This sets the result in tabular form so the descriptions all line up in the output:

```
#  cat::   Small domestic animal
#  +cat+:: Command to copy standard input
#          to standard output
```

For both kinds of labeled lists, if the body text starts on the same line as the label, then the start of that text determines the block indent for the rest of the body. The text may also start on the line following the label, indented from the start of the label. This is often preferable if the label is long. Both of the following are valid labeled list entries:

```
#  <tt>--output</tt> <i>name [, name]</i>::
#      specify the name of one or more output files. If multiple
#      files are present, the first is used as the index.
#
#  <tt>--quiet:</tt>:: do not output the names, sizes, byte counts,
#                      index areas, or bit ratios of units as
#                      they are processed.
```

Headings

Headings are entered on lines starting with equals signs. The more equals signs, the higher the level of heading:

```
#  = Level One Heading
#  == Level Two Heading
#  and so on...
```

Rules (horizontal lines) are entered using three or more hyphens:

```
#  and so it goes...
#  ----
#  The next section...
```

Documentation Modifiers

Method parameter lists are extracted and displayed with the method description. If a method calls yield, then the parameters passed to yield will also be displayed. For example, this code:

```
def fred
  # ...
  yield line, address
```

will be documented as follows:

```
fred() { |line, address| ... }
```

You can override this using a comment containing :yields: ... on the same line as the method definition:

```
def fred      # :yields: index, position
  # ...
  yield line, address
```